

**LAPORAN PRAKTIKUM  
ALGORITMA & STRUKTUR DATA  
MODUL 3**



**Single Linked List Circular dan Double Linked List Circular**

**Oleh:**

**Amalia Soraya**

**NIM. 2510817120002**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
MEI  
2026**

**LEMBAR PENGESAHAN**

**LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA**

**MODUL 3**

Laporan Praktikum Algoritma & Struktur Data Modul 3 : Single Linked List Circular dan Double Linked List Circular ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amalia Soraya  
NIM : 2510817120002

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani  
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D  
NIP. 198606132015041001

## DAFTAR ISI

|                         |     |
|-------------------------|-----|
| LEMBAR PENGESAHAN.....  | i   |
| DAFTAR ISI .....        | ii  |
| DAFTAR GAMBAR.....      | iii |
| DAFTAR TABEL .....      | iv  |
| SOAL 1 .....            | 1   |
| A. Source Code.....     | 1   |
| B. Pembahasan .....     | 5   |
| SOAL 2.....             | 9   |
| A. Source Code.....     | 10  |
| B. Output Program ..... | 11  |
| C. Pembahasan .....     | 11  |
| SOAL 3.....             | 13  |
| A. Source Code.....     | 13  |
| B. Pembahasan .....     | 17  |
| SOAL 4.....             | 21  |
| A. Source Code.....     | 22  |
| B. Output Program ..... | 24  |
| C. Pembahasan .....     | 24  |
| TAUTAN GIT .....        | 26  |

## **DAFTAR GAMBAR**

|                                                 |    |
|-------------------------------------------------|----|
| Gambar 1. Screenshot Hasil Jawaban Soal 2 ..... | 11 |
| Gambar 2. Screenshot Hasil Jawaban Soal 4 ..... | 24 |

## DAFTAR TABEL

|                                           |    |
|-------------------------------------------|----|
| Tabel 1. Source Code file stack.cpp ..... | 1  |
| Tabel 2. Source Code file prob1.cpp ..... | 10 |
| Tabel 3. Source Code file queue.cpp ..... | 13 |
| Tabel 4. Source Code file prob2.cpp ..... | 22 |

## SOAL 1

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

### A. Source Code

*Tabel 1. Source Code file `single_linked_list.cpp`*

|    |                                                                    |
|----|--------------------------------------------------------------------|
| 1  | <code>#include "single_linked_list.h"</code>                       |
| 2  | <code>#include &lt;iostream&gt;</code>                             |
| 3  | <code>using namespace std;</code>                                  |
| 4  |                                                                    |
| 5  | <code>void SingleLinkedList::init() {</code>                       |
| 6  | <code>    clear();</code>                                          |
| 7  | <code>    head = nullptr;</code>                                   |
| 8  | <code>    tail = nullptr;</code>                                   |
| 9  | <code>}</code>                                                     |
| 10 |                                                                    |
| 11 | <code>bool SingleLinkedList::is_empty() {</code>                   |
| 12 | <code>    if (head == nullptr &amp;&amp; tail == nullptr) {</code> |
| 13 | <code>        return true;</code>                                  |
| 14 | <code>    }</code>                                                 |
| 15 | <code>    return false;</code>                                     |
| 16 | <code>}</code>                                                     |
| 17 |                                                                    |
| 18 | <code>void SingleLinkedList::add_front(int val) {</code>           |
| 19 | <code>    // nambah di depan</code>                                |
| 20 | <code>    Node * newNode = new Node;</code>                        |
| 21 | <code>    if (is_empty()) {</code>                                 |
| 22 | <code>        head = tail = newNode;</code>                        |
| 23 | <code>        newNode-&gt;data = val;</code>                       |
| 24 | <code>        tail-&gt;next = head;</code>                         |
| 25 | <code>    } else {</code>                                          |
| 26 | <code>        newNode-&gt;data = val;</code>                       |
| 27 | <code>        newNode-&gt;next = head;</code>                      |
| 28 | <code>        head = newNode;</code>                               |

```

29         tail->next = head;
30     }
31     size++;
32 }
33
34 void SingleLinkedList::add_back(int val) {
35     Node * newNode = new Node;
36     newNode->data = val;
37     newNode->next = head;
38     if (is_empty()) {
39         add_front(val);
40         return;
41     } else {
42         tail->next = newNode;
43         tail = newNode;
44     }
45     size++;
46 }
47
48 void SingleLinkedList::add_idx(int val, int idx) {
49     if (idx==0) {
50         add_front(val);
51         return;
52     } else if (idx < 0) {
53         throw std::runtime_error("Woi input indeks yang
bener");
54     } else if (idx > size) {
55         throw std::runtime_error("KELEBIHAN
INDEKSNYA!");
56     } else if (idx == size) {
57         add_back(val);
58         return;
59     } else {
60         Node * newNode = new Node;
61         newNode->data = val;
62         Node * curr = head;
63         for (int i=0; i<idx-1; i++) {
64             curr = curr->next;
65         }
66         newNode->next = curr->next;
67         curr->next = newNode;
68     }
69     size++;
70 }
71
72 void SingleLinkedList::delete_front() {
73     if (is_empty()) return;

```

```

74     if (head == tail) {
75         delete head;
76         head = nullptr;
77         tail = nullptr;
78         size = 0;
79     } else {
80         Node * curr = head;
81         head = head->next;
82         tail->next = head;
83
84         delete curr;
85         size--;
86     }
87 }
88
89 void SingleLinkedList::delete_back() {
90     if (is_empty()) return;
91
92     if (head == tail) { //jika hanya 1 node
93         delete head;
94         head = tail = nullptr;
95         size--;
96     } else {
97         Node * curr = head;
98         while (curr->next != tail) {
99             curr = curr->next;
100         }
101         delete tail;
102         tail = curr;
103         tail->next = head;
104         size--;
105     }
106 }
107
108 void SingleLinkedList::delete_idx(int idx) {
109     if (is_empty()) return;
110     if (idx==0) {
111         delete_front();
112         return;
113     }
114     Node * curr = head;
115     for (int i=0; i<idx-1; i++) {
116         curr = curr->next;
117     }
118     Node * temp = curr->next;
119     curr->next = temp->next;
120

```



```

121     if (temp == tail) tail = curr; //memindah tail
122
123     delete temp;
124     size--;
125 }
126
127 void SingleLinkedList::display() {
128     if (!is_empty()) {
129         Node * curr;
130         curr = head;
131         do {
132             cout << curr->data << " ";
133             curr = curr->next;
134         } while (curr != head);
135
136         cout << endl;
137     } else {
138         cout << "List Kosong" << endl;
139     }
140 }
141
142 int SingleLinkedList::get(int idx) {
143     if (is_empty()) throw std::runtime_error("List
kosong");
144
145     Node * curr = head;
146     for (int i=0; i<idx; i++) {
147         curr = curr->next;
148     }
149     return curr->data;
150 }
151
152 void SingleLinkedList::set(int val, int idx) {
153     if (is_empty()) return;
154
155     Node * curr = head;
156     for (int i=0; i<idx; i++) {
157         curr = curr->next;
158     }
159     curr->data = val;
160 }
161
162 void SingleLinkedList::clear() {
163     while (!is_empty()) {
164         Node * curr = head;
165         if (size == 1) { head = tail = nullptr; }
166         else {

```

|     |                    |
|-----|--------------------|
| 167 | head = head->next; |
| 168 | tail->next = head; |
| 169 | }                  |
| 170 | delete curr;       |
| 171 | size--;            |
| 172 | }                  |
| 173 | head = nullptr;    |
| 174 | tail = nullptr;    |
| 175 | size = 0;          |
| 176 | }                  |

## B. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `single_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 5 sampai 9 adalah fungsi inisialisasi. Gunanya untuk inisialisasi pointer head dan tail. Cara kerjanya yaitu menggunakan fungsi clear dulu agar benar benar tidak ada linked list sebelumnya, lalu inisialisasi pointer head dan tail di nullptr (pointer kosong) karena belum ada linked list. Kompleksitas waktu fungsi ini adalah  $O(1)$  karena fungsi berjalan statis tanpa dipengaruhi nilai variabel lain. Kompleksitas ruang fungsi ini adalah  $O(1)$  karena tidak ada variabel array yang bergantung pada nilai ukuran array.

Pada baris 11 sampai 16 adalah fungsi `is_empty`. Gunanya untuk mengecek apakah linked list kosong atau tidak. Cara kerjanya dengan mengecek apakah head dan tail menunjuk ke nullptr? Jika iya, nilainya true, jika tidak, nilainya false.

Pada baris 18 sampai 32 adalah fungsi `add_front`. Gunanya untuk menambahkan node di depan linked list. Cara kerjanya dimulai dari menambah node baru di variabel `newNode`. Lalu mengecek apakah linked list kosong atau tidak. Kalau kosong, node baru akan ditunjuk oleh head dan tail, lalu menambahkan nilai yang diinput ke data node baru. Agar menjadi circular, pointer next si node baru menunjuk ke head. Kalau linked list tidak kosong, node baru diisi nilai, lalu mengatur pointer next si node baru ke head, lalu pointer head dipindah ke node baru, dan next nya tail diarahkan ke head yang baru (node baru tadi). Kompleksitas ruangnya  $O(1)$  karna variabelnya konstan dan kompleksitas waktunya  $O(1)$  karna program berjalan konstan tanpa ada loop.

Pada baris 34 sampai 46 adalah fungsi `add_back`. Gunanya untuk menambahkan node di belakang (node terakhir) linked list. Cara kerjanya dengan membuat pointer `newNode`

dulu, lalu menambahkan nilai ke data node baru, lalu mengatur pointer next nya node baru untuk menunjuk ke head. Kalau linked list kosong, jalankan fungsi `add_front`. Kalau tidak kosong, atur tail next untuk menunjuk ke node baru dan barulah pointer tail dipindah ke node baru. Kompleksitas ruangnya  $O(1)$  karna menambah 1 node aja dan kompleksitas waktunya  $O(1)$  karna tidak ada looping. Tapi kalau fungsi ini terus dipanggil, kompleksitas ruangnya jadi  $O(n)$  karna linked list emang linear kompleksitasnya.

Pada baris 48 sampai 70 adalah fungsi `add_idx`. Gunanya untuk menambahkan node di indeks tertentu, disisipkan diantara node lain. Cara kerjanya pertama ngecek apakah indeksnya 0? Kalau iya, panggil fungsi `add_front`. Lalu apakah indeks kurang dari 0? Maka throw pesan error. Lalu apakah indeks lebih dari size linkedlist? Kalau iya maka throw pesan error out of index. Lalu apakah nilai indeks sama dengan nilai size? Kalau iya, berarti ini nilai terakhir linked list, jadi jalankan fungsi `add_back` saja. Lalu kalau memang indeks ada pada sesudah head dan sebelum tail, maka buat node baru dan masukkan nilai ke node baru. Lalu buat pointer curr menunjuk ke head, di traverse dengan loop hingga indeks-1. Lalu atur pointer next si node baru menunjuk ke node sesudah curr, lalu next nya curr menunjuk ke node baru tadi. Sehingga node baru berada diantara curr dan `curr->next` itu. Kompleksitas ruangnya adalah  $O(n)$  karna ini membuat linked list dan kompleksitas waktunya  $O(n)$  karna melakukan traverse dengan loop.

Pada baris 72 sampai 87 adalah fungsi `delete_front`. Gunanya agar bisa menghapus node paling depan si linked list. Cara kerjanya pertama tu fungsi mengecek apakah linked list kosong atau tidak (kalau kosong, kembali). Lalu mengecek apakah nilai head sama dengan tail (apakah cuma ada 1 node)? Kalau iya, hapus node head dan jadikan head dan tail menunjuk ke nullptr serta ubah size menjadi 0. Kalau node tidak satu-satunya di linked list, buat node curr yang menunjuk ke head lalu geser pointer head ke sesudah si head ini lalu ubah next nya si tail agar menunjuk ke head baru. Setelah memindah-mindah pointer, baru node curr yang menunjuk di depan tadi dihapus dan kurangi size. Kompleksitas ruangnya  $O(1)$  karna ruang yang digunakan ketika fungsi ini berjalan itu konstan dan kompleksitas waktu juga  $O(1)$  karna tidak ada looping.

Pada baris 89 sampai 106 adalah fungsi `delete_black`. Gunanya agar bisa menghapus node paling belakang. Cara kerjanya pertama itu ngecek apakah linked list kosong atau tidak (kalau iya, kembalikan/gausah jalan fungsinya). Lalu mengecek apakah Cuma ada 1 node?

Kalau iya, hapus head, ubah head dan tail menunjuk nullptr, kurangi size. Jika node lebih dari 1 tadi, masuk ke else. Buat node curr yang menunjuk ke head, lalu traverse sampai sebelum tail, lalu hapus tail, lalu geser tail ke curr, lalu next nya si tail diubah menunjuk ke head. Lalu kurangi size. Kompleksitas ruangnya  $O(n)$  karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya  $O(n)$  karna ada traverse dengan loop.

Pada baris 108 sampai 125 adalah fungsi `delete_idx`. Gunanya agar bisa menghapus node tertentu yang ada di tengah linkedlist (setelah head, sebelum tail). Cara kerjanya pertama mengecek apakah linked list kosong atau tidak (kalau kosong ya kembali). Lalu kalau idxnya 0, jalankan `delete_front`. Kalau tidak keduanya, fungsinya jalan mulai dari membuat node curr yang akan di traverse. Lalu node baru \*temp yang menunjuk sesudah node curr. Sebelum menghapus \*temp, nextnya curr dipindah ke node sesudah temp. Misalnya kebetulan temp nya ini ternyata tail, maka tailnya digeser ke curr dulu. Lanjut menghapus temp dan mengurangi size. Kompleksitas ruangnya  $O(n)$  karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya  $O(n)$  karna ada traverse dengan loop.

Pada baris 127 sampai 140 adalah fungsi `display`. Gunanya agar bisa menampilkan seluruh nilai linked list. Berjalan kalau linked list tidak kosong, kalau kosong cetak "list kosong". Nah kalau tidak kosong, buat node curr untuk traverse. Selama traverse, cetak juga isi data si curr. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan menyimpan 1 nilai dan kompleksitas waktunya adalah  $O(n)$  karna brute force untuk mencetak nilai data setiap node.

Pada baris 142 sampai 150 adalah fungsi `get`. Gunanya agar bisa mengambil nilai yang ada dalam data node. Cara kerjanya, pertama mengecek apakah linked list kosong atau tidak, kalau kosong throw pesan error. Kalau tidak, buat node curr untuk traverse sampai indeks yang diminta. Kalau sudah, kembalikan nilai data dalam node curr. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan menyimpan 1 nilai dan kompleksitas waktunya adalah  $O(n)$  karna traverse mencari node idx.

Pada baris 152 sampai 160 adalah fungsi `set`. Gunanya agar bisa mengganti nilai dalam data node tertentu. Cara kerjanya pertama mengecek linked list kosong atau tidak, kalau kosong return. Kalau tidak, buat node curr dan traverse. Sesudah traverse, ganti nilai

data di curr dengan nilai baru. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan dan kompleksitas waktunya adalah  $O(n)$  karna traverse hingga node idx.

Pada baris 162 sampai 176 adalah fungsi clear. Gunanya untuk membersihkan atau menghapus semua node linked list. Selama linked list tidak kosong, traverse berjalan dengan membuat node curr setiap iterasi namun juga menghapus curr tersebut setiap iterasi. Kalau cuma ada 1 node, langsung nullptr kan saja head dan tailnya. Terakhir, ubah head dan tail menunjuk ke nullptr dan ubah size menjadi 0. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan dan kompleksitas waktunya adalah  $O(n)$  karna brute force untuk menghapus setiap node.

## SOAL 2

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh  $N$  buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh  $K$  langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ( $K = K + 1$ ).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ( $K = K - 1$ ).
- Jika pada suatu titik energi lompatan habis atau negatif ( $K \leq 0$ ), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai  $K$  awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

### Batasan

- $1 \leq N \leq 10^4$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$  (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1

### Format Input

$N$   $K$

$A_1$   $A_2$  ...  $A_N$

### Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

## Contoh Kasus

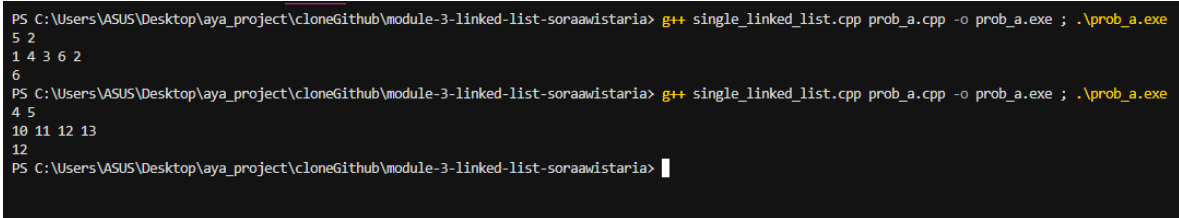
| Input              | Output |
|--------------------|--------|
| 5 2<br>1 4 3 6 2   | 6      |
| 4 5<br>10 11 12 13 | 12     |

### A. Source Code

*Tabel 2. Source Code file prob\_a.cpp*

|    |                                 |
|----|---------------------------------|
| 1  | #include "single_linked_list.h" |
| 2  | #include <iostream>             |
| 3  | using namespace std;            |
| 4  |                                 |
| 5  | int main () {                   |
| 6  | SingleLinkedList sll;           |
| 7  | sll.init();                     |
| 8  |                                 |
| 9  | int N, K, x;                    |
| 10 | cin >> N >> K;                  |
| 11 |                                 |
| 12 | for (int i = 0; i < N; i++) {   |
| 13 | cin >> x;                       |
| 14 | sll.add_back(x);                |
| 15 | }                               |
| 16 | int k_awal = K;                 |
| 17 | int idx = 0;                    |
| 18 | while (sll.size > 1) {          |
| 19 | idx = (idx + K - 1) % sll.size; |
| 20 |                                 |
| 21 | if (sll.get(idx) % 2 == 0) {    |
| 22 | K++;                            |
| 23 | } else {                        |
| 24 | K--;                            |
| 25 | }                               |
| 26 | sll.delete_idx(idx);            |
| 27 |                                 |
| 28 | if (K <= 0) K = k_awal;         |
| 29 |                                 |
| 30 | idx = idx % sll.size;           |
| 31 | }                               |
| 32 | sll.display();                  |
| 33 | }                               |

## B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; .\prob_a.exe
5
2
1 4 3 6 2
6
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; .\prob_a.exe
4 5
10 11 12 13
12
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria>
```

Gambar 1. Screenshot Hasil Jawaban Soal 2

## C. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `single_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 6 adalah deklarasi struct single linked list dengan nama `sll`. Gunanya untuk mendeklarasi struct agar variabel dan fungsi di dalam struct bisa digunakan untuk kode program soal 2. Baris 7 adalah pemanggilan fungsi `init`, karna menginisialisasi pointer `head` dan `tail` sebelum dipakai dalam program.

Pada baris 9 digunakan untuk deklarasi variabel yang akan digunakan dalam program. Baris 10 digunakan untuk input user, cara kerjanya dengan meminta input nilai dari user, lalu hasil input user disimpan ke variabel `N` (total nilai) dan `K` (loncatan batu).

Pada baris 12 sampai 15 adalah looping, digunakan agar mendapatkan input user secara berulang dan juga digunakan untuk menambahkan data input user ke linked list. Cara kerjanya yaitu program akan iterasi sebanyak `N` kali, dan sebanyak `N` kali ini user harus menginputkan nilai. Nilai input masuk ke variabel `x`. Baris 14 digunakan untuk memanggil fungsi `add_back`, cara kerjanya fungsi harus diisi argumen (variabel `x`) agar nilai variabel `x` masuk ke linked list.

Pada baris 16 dan 17 adalah inisialisasi variabel. Baris 16 digunakan untuk menyimpan nilai `K` untuk dipakai pada program, baris 17 digunakan untuk inisialisasi indeks 0 agar bisa menghitung indeks traverse.

Pada baris 18 sampai 31 adalah perulangan selama batu relikui masih lebih dari 1. Digunakan untuk traverse berulang-ulang hingga batu relikui tersisa 1 dan untuk menghapus node yang menjadi target. Cara kerjanya yaitu menginisialisasi indeks node yang akan



dihapus ( $\text{idx} + K - 1$  untuk mendapatkan indeks node, modulo agar circular). Lalu mengecek apakah data pada node tersebut nilainya genap atau ganjil. Kalau genap, nilai  $K$  ditambah 1, kalau ganjil nilai  $K$  dikurangi 1. Setelah itu node tersebut dihapus (batu dihancurkan).

Pada baris 28 digunakan untuk menghindari nilai  $K$  menjadi minus dengan mengembalikan nilai  $K$  ke nilai  $K$  awal jika minus. Baris 30 digunakan untuk menggunakan lagi nilai  $\text{idx}$ . Baris 32 digunakan untuk menampilkan nilai node yang tersisa. Kompleksitas ruang program ini adalah  $O(n)$  karena ada linked list dan kompleksitas waktunya  $O(n^2)$  karena ada while yang mengulang-ulang iterasi hingga node tersisa 1 dan di dalam while ada fungsi `delete_idx` yang juga menggunakan while loop ketika berjalan.

### SOAL 3

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

#### A. Source Code

*Tabel 3. Source Code file `double_linked_list.cpp`*

|    |                                                                    |
|----|--------------------------------------------------------------------|
| 1  | <code>#include "double_linked_list.h"</code>                       |
| 2  | <code>#include &lt;iostream&gt;</code>                             |
| 3  | <code>using namespace std;</code>                                  |
| 4  |                                                                    |
| 5  | <code>void DoubleLinkedList::init() {</code>                       |
| 6  | <code>    clear();</code>                                          |
| 7  | <code>    head = nullptr;</code>                                   |
| 8  | <code>    tail = nullptr;</code>                                   |
| 9  | <code>}</code>                                                     |
| 10 |                                                                    |
| 11 | <code>bool DoubleLinkedList::is_empty() {</code>                   |
| 12 | <code>    if (head == nullptr &amp;&amp; tail == nullptr) {</code> |
| 13 | <code>        return true;</code>                                  |
| 14 | <code>    }</code>                                                 |
| 15 | <code>    return false;</code>                                     |
| 16 | <code>}</code>                                                     |
| 17 |                                                                    |
| 18 | <code>void DoubleLinkedList::add_front(char val) {</code>          |
| 19 | <code>    //tambah deoan</code>                                    |
| 20 | <code>    Node * newNode = new Node;</code>                        |
| 21 | <code>    newNode-&gt;data = val;</code>                           |
| 22 | <code>    if (is_empty()) {</code>                                 |
| 23 | <code>        head = tail = newNode;</code>                        |
| 24 | <code>        tail-&gt;next = head;</code>                         |
| 25 | <code>        tail-&gt;prev = newNode;</code>                      |
| 26 | <code>    } else {</code>                                          |
| 27 | <code>        newNode-&gt;next = head;</code>                      |
| 28 | <code>        head = newNode;</code>                               |

```

29
30         tail->next = head;
31         newNode->prev = tail;
32     }
33     size++;
34 }
35
36 void DoubleLinkedList::add_back(char val) {
37     if (is_empty()) {
38         add_front(val);
39         return;
40     }
41     Node * newNode = new Node;
42     newNode->data = val;
43     newNode->next = head;
44     newNode->prev = tail;
45
46     tail->next = newNode;
47     tail = newNode;
48     head->prev = tail;
49     size++;
50 }
51
52 void DoubleLinkedList::add_idx(char val, int idx) {
53     //insert
54     if (idx == 0) {
55         add_front(val);
56         return;
57     } else if (idx < 0) {
58         throw std::runtime_error("Woi input indeks yang
bener");
59     } else if (idx > size) {
60         throw std::runtime_error("KELEBIHAN
INDEKSNYA!");
61     } else if (idx == size) {
62         add_back(val);
63         return;
64     } else {
65         Node * newNode = new Node;
66         newNode->data = val;
67         Node * curr = head;
68         for (int i=0; i < idx-1; i++) {
69             curr = curr->next;
70         }
71         newNode->next = curr->next;
72         newNode->prev = curr;
73         curr->next = newNode;

```

```

74         curr->next->prev = newNode;
75     }
76     size++;
77 }
78
79 void DoubleLinkedList::delete_front() {
80     //hapus depan
81     if (is_empty()) return;
82     if (head == tail) {
83         delete head;
84         head = nullptr;
85         tail = nullptr;
86         size = 0;
87     } else {
88         Node * curr = head;
89         head = head->next;
90         head->prev = tail;
91         tail->next = head;
92
93         delete curr;
94         size--;
95     }
96 }
97
98 void DoubleLinkedList::delete_back() {
99     if (is_empty()) return;
100    if (head == tail) {
101        delete_front();
102        return;
103    } else {
104        Node * curr = head;
105        while (curr->next != tail) {
106            curr = curr->next;
107        }
108        delete tail;
109        tail = curr;
110        tail->next = head;
111        head->prev = tail;
112        size--;
113    }
114 }
115
116 void DoubleLinkedList::delete_idx(int idx) {
117     if (is_empty()) return;
118     Node * nodePtr = head;
119     int count = 0;
120

```

```

121     if (idx==0) {
122         delete_front();
123         return;
124     }
125     if (nodePtr->next == head) {
126         delete_back ();
127         return;
128     }
129     while (count < idx-1 && nodePtr->next != head) {
130         nodePtr = nodePtr->next;
131         count++;
132     }
133     Node * target = nodePtr->next;
134     nodePtr->next = target->next;
135     target->next->prev = nodePtr;
136
137     delete target;
138     size--;
139 }
140
141 void DoubleLinkedList::display() {
142     if (!is_empty()) {
143         Node * curr;
144         curr = head;
145         do {
146             cout << curr->data;
147             curr = curr->next;
148         } while (curr != head);
149
150         cout << endl;
151     } else {
152         cout << "List Kosong" << endl;
153     }
154 }
155
156 char DoubleLinkedList::get(int idx) {
157     if (is_empty()) throw std::runtime_error("List
kosong");
158
159     Node * curr = head;
160     for (int i=0; i<idx; i++) {
161         curr = curr->next;
162     }
163     return curr->data;
164 }
165
166 void DoubleLinkedList::set(char val, int idx) {

```

```

167     if (is_empty()) return;
168
169     Node * curr = head;
170     for (int i=0; i<idx; i++) {
171         curr = curr->next;
172     }
173     curr->data = val;
174 }
175
176 void DoubleLinkedList::clear() {
177     while (!is_empty()) {
178         Node * curr = head;
179         if (size == 1) { head = tail = nullptr; }
180         else {
181             head = head->next;
182             tail->next = head;
183         }
184         delete curr;
185         size--;
186     }
187     head = nullptr;
188     tail = nullptr;
189     size = 0;
190 }

```

## B. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `single_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 5 sampai 9 adalah fungsi inisialisasi (`init`). Gunanya untuk inisialisasi pointer `head` dan `tail`. Cara kerjanya yaitu menggunakan fungsi `clear` dulu agar benar benar tidak ada linked list sebelumnya, lalu inisialisasi pointer `head` dan `tail` di `nullptr` (pointer kosong) karena belum ada linked list. Kompleksitas waktu fungsi ini adalah  $O(1)$  karena fungsi berjalan statis tanpa dipengaruhi nilai variabel lain. Kompleksitas ruang fungsi ini adalah  $O(1)$  karena tidak ada variabel array yang bergantung pada nilai ukuran array.

Pada baris 11 sampai 16 adalah fungsi `is_empty`. Gunanya untuk mengecek apakah linked list kosong atau tidak. Cara kerjanya dengan mengecek apakah `head` dan `tail` menunjuk ke `nullptr`? Jika iya, nilainya `true`, jika tidak, nilainya `false`.

Pada baris 18 sampai 34 adalah fungsi `add_front`. Gunanya untuk menambahkan node di depan linked list. Cara kerjanya dimulai dari menambah node baru di variabel `newNode` dan memasukkan nilai `val` ke data `newNode`. Lalu mengecek apakah linked list kosong atau tidak. Kalau kosong, node baru akan ditunjuk oleh `head` dan `tail`. Agar menjadi circular, pointer `next` si `tail` baru menunjuk ke `head` dan pointer `prev` si `tail` menunjuk ke `newNode`. Kalau linked list tidak kosong, mengatur pointer `next` si node baru menunjuk ke `head`, lalu pointer `head` dipindah ke node baru, dan `next` nya `tail` diarahkan ke `head` yang baru (node baru tadi), serta mengisi nilai `prev` si node baru untuk menunjuk ke `tail` agar circular. Kompleksitas ruangnya  $O(1)$  karna variabelnya konstan dan kompleksitas waktunya  $O(1)$  karna program berjalan konstan tanpa ada loop.

Pada baris 36 sampai 50 adalah fungsi `add_back`. Gunanya untuk menambahkan node di belakang (node terakhir) linked list. Cara kerjanya dengan mengecek apakah linked list kosong atau tidak (kalau kosong, gunakan fungsi `add_front`). Lalu membuat pointer `newNode` dulu, lalu menambahkan nilai ke data node baru, lalu menggeser pointer `next` nya node baru untuk menunjuk ke `head` dan mengubah `prev`nya ke menunjuk `tail`. Kemudian geser `tail next` untuk menunjuk ke node baru dan barulah pointer `tail` dipindah ke node baru dan `prev` dari `head` diubah menunjuk ke `tail` yang sekarang. Kemudian `size` ditambah. Kompleksitas ruangnya  $O(1)$  karna menambah 1 node aja dan kompleksitas waktunya  $O(1)$  karna tidak ada looping.

Pada baris 52 sampai 77 adalah fungsi `add_idx`. Gunanya untuk menambahkan node di indeks tertentu, disisipkan diantara node lain. Cara kerjanya pertama mengecek apakah indeksnya 0? Kalau iya, panggil fungsi `add_front`. Lalu apakah indeks kurang dari 0? Maka throw pesan error. Lalu apakah indeks lebih dari `size linkedlist`? Kalau iya maka throw pesan error out of index. Lalu apakah nilai indeks sama dengan nilai `size`? Kalau iya, berarti ini nilai terakhir linked list, jadi jalankan fungsi `add_back` saja. Lalu kalau memang indeks ada pada sesudah `head` dan sebelum `tail`, maka buat node baru dan masukkan nilai ke node baru. Lalu buat pointer `curr` menunjuk ke `head`, di traverse dengan loop hingga indeks-1. Lalu atur pointer `next` si node baru menunjuk ke node sesudah `curr` dan `prev` nya node baru menunjuk ke `curr`. Lalu `next` nya `curr` menunjuk ke node baru tadi. Sehingga node baru berada diantara `curr` dan `curr->next` itu, serta `prev`nya si `curr->next` diarahkan ke node `newNode`.

Kompleksitas ruangnya adalah  $O(n)$  karna ini membuat linked list dan kompleksitas waktunya  $O(n)$  karna melakukan traverse dengan loop.

Pada baris 79 sampai 96 adalah fungsi `delete_front`. Gunanya agar bisa menghapus node paling depan si linked list. Cara kerjanya pertama tu fungsi mengecek apakah linked list kosong atau tidak (kalau kosong, kembali). Lalu mengecek apakah nilai head sama dengan tail (apakah cuma ada 1 node)? Kalau iya, hapus node head dan jadikan head dan tail menunjuk ke nullptr serta ubah size menjadi 0. Kalau node tidak satu-satunya di linked list, buat node curr yang menunjuk ke head lalu geser pointer head ke sesudah si head ini lalu ubah next nya si tail agar menunjuk ke head baru. Tambahan, prev nya head diarahkan ke tail. Setelah memindah-mindah pointer, baru node curr yang menunjuk di depan tadi dihapus dan kurangi size. Kompleksitas ruangnya  $O(1)$  karna ruang yang digunakan ketika fungsi ini berjalan itu konstan dan kompleksitas waktu juga  $O(1)$  karna tidak ada looping.

Pada baris 98 sampai 114 adalah fungsi `delete_back`. Gunanya agar bisa menghapus node paling belakang. Cara kerjanya pertama itu ngecek apakah linked list kosong atau tidak (kalau iya, kembalikan/gausah jalan fungsinya). Lalu mengecek apakah Cuma ada 1 node? Kalau iya, panggil fungsi `delete_front`. Jika node lebih dari 1 tadi, masuk ke else. Buat node curr yang menunjuk ke head, lalu traverse sampai sebelum tail, lalu hapus tail, lalu geser tail ke curr, lalu next nya si tail diubah menunjuk ke head dan prev nya head menunjuk ke tail. Lalu kurangi size. Kompleksitas ruangnya  $O(n)$  karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya  $O(n)$  karna ada traverse dengan loop.

Pada baris 116 sampai 139 adalah fungsi `delete_idx`. Gunanya agar bisa menghapus node tertentu yang ada di tengah linkedlist (setelah head, sebelum tail). Cara kerjanya pertama mengecek apakah linked list kosong atau tidak (kalau kosong ya kembali). Lalu kalau idxnya 0, jalankan `delete_front`. Kalau tidak keduanya, fungsinya jalan mulai dari membuat node `nodePtr` yang akan di traverse dan variabel count untuk parameter while. Lalu traverse `nodePtr` dengan while sampai indeks-1 dan tidak sampai tail. Setelah traverse, buat node target yang menunjuk ke node sesudah `nodePtr`, lalu next nya `nodePtr` diarahkan ke node sesudah target, lalu prev nya si node sesudah node target tadi diarahkan ke `nodePtr` (jadi, target ada di antara `nodePtr` dan `target->next`). Setelah itu, node target dihapus dan size dikurangi. Kompleksitas ruangnya  $O(n)$  karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya  $O(n)$  karna ada traverse dengan loop.



Pada baris 141 sampai 154 adalah fungsi display. Gunanya agar bisa menampilkan seluruh nilai linked list. Berjalan kalau linked list tidak kosong, kalau kosong cetak "list kosong". Nah kalau tidak kosong, buat node curr untuk traverse. Selama traverse, cetak juga isi data si curr. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan menyimpan 1 nilai dan kompleksitas waktunya adalah  $O(n)$  karna brute force untuk mencetak nilai data setiap node.

Pada baris 156 sampai 164 adalah fungsi get. Gunanya agar bisa mengambil nilai yang ada dalam data node. Cara kerjanya, pertama mengecek apakah linked list kosong atau tidak, kalau kosong throw pesan error. Kalau tidak, buat node curr untuk traverse sampai indeks yang diminta. Kalau sudah, kembalikan nilai data dalam node curr. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan menyimpan 1 nilai dan kompleksitas waktunya adalah  $O(n)$  karna traverse mencari node idx.

Pada baris 166 sampai 174 adalah fungsi set. Gunanya agar bisa mengganti nilai dalam data node tertentu. Cara kerjanya pertama mengecek linked list kosong atau tidak, kalau kosong return. Kalau tidak, buat node curr dan traverse. Sesudah traverse, ganti nilai data di curr dengan nilai baru. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan dan kompleksitas waktunya adalah  $O(n)$  karna traverse hingga node idx.

Pada baris 176 sampai 190 adalah fungsi clear. Gunanya untuk membersihkan atau menghapus semua node linked list. Selama linked list tidak kosong, traverse berjalan dengan membuat node curr setiap iterasi namun juga menghapus curr tersebut setiap iterasi. Kalau cuma ada 1 node, langsung nullptr kan saja head dan tailnya. Terakhir, ubah head dan tail menunjuk ke nullptr dan ubah size menjadi 0. Kompleksitas ruangnya adalah  $O(1)$  karna variabel curr konstan dan kompleksitas waktunya adalah  $O(n)$  karna brute force untuk menghapus setiap node.

## SOAL 4

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi  $N$  karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama  $R$  ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak  $X$  langkah. Proyektor Beta dipaksa melompat mundur sebanyak  $Y$  langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah  $R$  ronde selesai dieksekusi.

### Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Keterangan:

- $n$ : jumlah elemen
- $k$ : ukuran jendela
- $A_i$ : elemen array

### Format Input

N R

C1 C2 ... CN

X1 Y1

X2 Y2

...

XR YR

Keterangan:  $C_i$  adalah satu karakter char tunggal.

## Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

## Contoh Kasus

| Input                                           | Output |
|-------------------------------------------------|--------|
| 4 2<br>A B C D<br>1 1<br>2 3                    | ACB    |
| 5 3<br>V W X Y Z<br>100000 100000<br>2 2<br>5 5 | VWYZ   |

## A. Source Code

Tabel 4. Source Code file prob\_b.cpp

|    |                                 |
|----|---------------------------------|
| 1  | #include "double_linked_list.h" |
| 2  | #include <iostream>             |
| 3  | using namespace std;            |
| 4  |                                 |
| 5  | int main() {                    |
| 6  | DoubleLinkedList dll;           |
| 7  | int N, R;                       |
| 8  | char C;                         |
| 9  | dll.init();                     |
| 10 |                                 |

```

11     cin >> N >> R;
12     for (int i=0; i < N; i++) {
13         cin >> C;
14         dll.add_back(C);
15     }
16     int X[R], Y[R];
17     for (int i=0; i<R; i++) {
18         cin >> X[i] >> Y[i];
19     }
20
21     cout << endl;
22     Node * alpha = dll.head;
23     Node * beta = dll.tail;
24     for (int i=0; i<R; i++) {
25
26         for (int j=0; j < X[i] ; j++) {
27             alpha = alpha->next;
28         }
29         for (int k=0; k < Y[i] ;k++) {
30             beta = beta->prev;
31         }
32
33         if (alpha->next == beta || alpha->prev == beta)
34     {
35         char swap_beta = beta->data;
36         beta->data = alpha->data;
37         alpha->data = swap_beta;
38     } else if (alpha == beta) {
39         Node * target = dll.head;
40         int idx = 0;
41
42         while (target != alpha) {
43             target = target->next;
44             idx++;
45         }
46         alpha = target->next;
47         beta = target->prev;
48         dll.delete_idx(idx);
49     }
50     dll.display();
51 }

```

## B. Output Program

```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ double_linked_list.cpp prob_b.cpp -o prob_b.exe ; .\prob_b.exe
4 2
A B C D
1 1
2 3

ACB
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ double_linked_list.cpp prob_b.cpp -o prob_b.exe ; .\prob_b.exe
5 3
V W X Y Z
100000 100000
2 2
5 5

VWYZ
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> |
```

Gambar 2. Screenshot Hasil Jawaban Soal 4

## C. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `double_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 6 adalah deklarasi struct double linked list dengan nama `dll`. Gunanya untuk mendeklarasi struct agar variabel dan fungsi di dalam struct bisa digunakan untuk kode program soal 2. Baris 7 digunakan untuk mendeklarasi variabel `N` (untuk jumlah char) dan `R` (ronde putar). Baris 8 digunakan untuk deklarasi variabel `C` bertipe char. Baris 9 digunakan untuk inisialisasi pointer dalam double linked list dengan memanggil fungsi `init` yang ada dalam struct. Pada baris 11 digunakan untuk input user, lalu nilai input itu dimasukkan ke variabel `N` dan `R`.

Pada baris 12 sampai 15 adalah loop yang digunakan untuk mengambil nilai input user dan memasukkannya ke linked list dengan fungsi `add_back` pada double linked list. Baris 16 digunakan untuk deklarasi array `X` dan `R`, lalu baris 17 sampai 19 adalah loop untuk input user dan nilainya akan disimpan dalam array `X` (lompatan alpha) dan `R` (lompatan beta).

Pada baris 22 dan 23 adalah inisialisasi pointer `alpha` dan `beta` untuk menunjuk ke head dan tail linked list. Lalu baris 24 sampai 49 adalah inti jawaban soal. Cara kerjanya dimulai dari iterasi luar (`i`) untuk perputaran sebanyak `R` ronde, lalu selama `R` ronde ini ada iterasi `j` untuk traverse pointer `alpha` sampai `X[i]` dan iterasi `k` untuk traverse pointer `beta` sampai `Y[i]`. Traverse hingga nilai yang sudah diinput user tadi.

Pada baris 33 sampai 47 adalah penentu tindakan proyektor alpha dan beta. Jika alpha dan beta bersebelahan (setelah alpha ada beta atau sebelum alpha ada beta), maka swap nilai alpha dan beta dengan menyimpan dulu nilai beta pada variabel `swap_beta` dan nilai data beta diubah menjadi alpha dan nilai alpha diubah menjadi beta sebelumnya dari `swap_beta`. Jika alpha dan beta menunjuk node yang sama, maka hapus node tersebut dengan cara traverse pointer target hingga bertemu pointer alpha dan selama traverse itu nilai `idx` ditambah untuk mendapatkan indeks target node yang akan dihapus (karena fungsi `delete` memerlukan nilai integer indeks). Setelah traverse hingga alpha dan menambah increment `idx`, pointer alpha dipindah menunjuk ke node sesudah target dan pointer beta dipindah menunjuk ke node sebelum target karna node target akan dihapus. Lalu pada baris 47, pemanggilan fungsi untuk menghapus node `idx`. Terakhir, pemanggilan fungsi `display` untuk menampilkan sisa atau hasil node sesudah operasi tadi.

Kompleksitas ruang pada fungsi ini adalah  $O(N + R)$  karena memakai linked list dan array untuk nilai `X[R]` dan `Y[R]`. Kompleksitas waktunya adalah  $O(n^2)$  karena nested loop, iterasi luar yang bergantung pada nilai `R` dan iterasi dalam yang bergantung pada `X[i]`, `Y[i]`, dan `idx` pada while loop.

## TAUTAN GIT

Repository : <https://github.com/DSA26-ULM/module-3-linked-list-soraawistaria>